



UNIVERSIDAD
COMPLUTENSE
MADRID

TAREA DISEÑO DE INGESTAS Y LAGOS DE DATOS

Autor: Alejandro de Cora

Actualizado: 02-09-2026

- El código completo del trabajo se encuentra alojado en el repositorio:
<https://github.com/adecora/lakehouses-ucm>.
- Fuentes utilizadas:
 - El material de la asignatura **Diseño de ingestas y data lakehouses**.
 - [Documentación de Azure Databricks](#).
 - [Databricks Learning Library](#).
 - [Tutorial de Pydantic](#).
 - Modelos de lenguaje consultados:
 - Gemini Pro Latest
 - Gemini Flash Latest

Índice

Parte 1: Diseño de la arquitectura del Data Lakehouse

Revisión inicial

Arquitectura lakehouse

Parte 2: Implementación del motor de ingesta

Paquete python: "motor_ingesta"

Pipeline: "lakehouses_ucm_etl"

Job: "lakehouses_ucm"

Evidencias de ejecución

Referencias y Bibliografía



Farm IA

agricultura inteligente basada en datos

Parte 1: Diseño de la arquitectura del Data Lakehouse

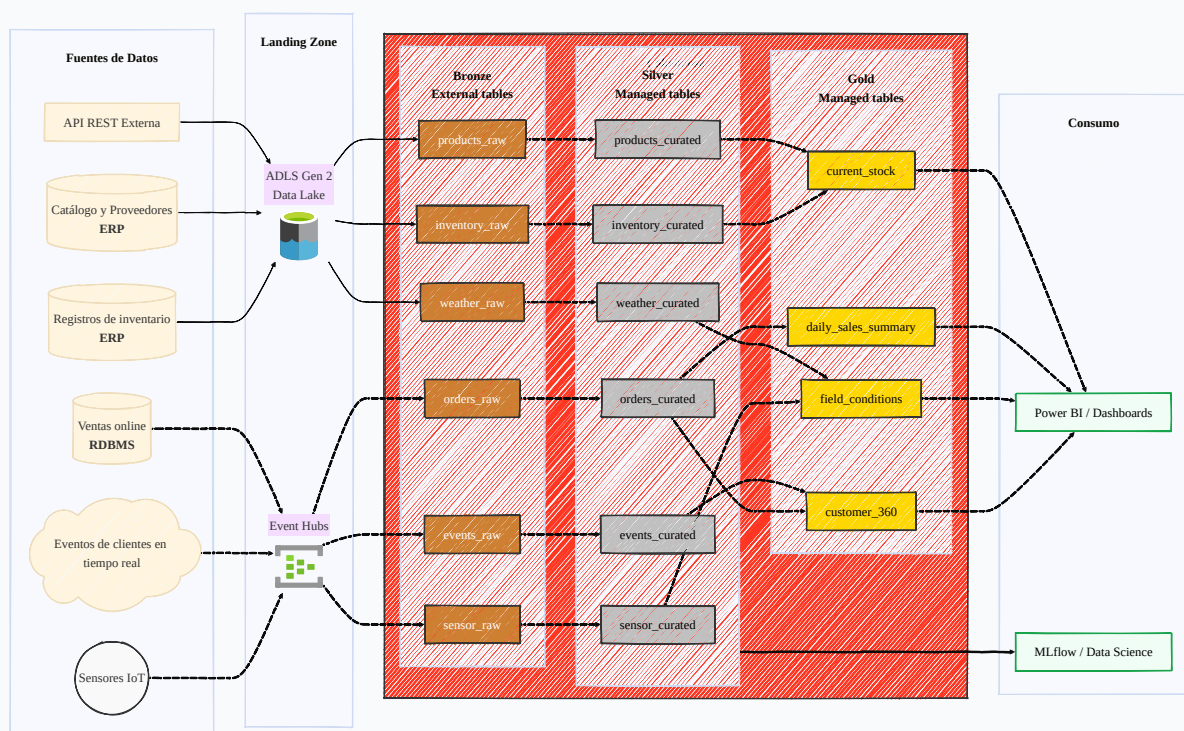
Revisión inicial

Antes de realizar el diseño de la arquitectura, se lleva a cabo un estudio previo de las fuentes de datos de **FarmIA**. Para ello, se parte de las fuentes mencionadas en la tarea y de los [datasets sintéticos generados en el repositorio de la asignatura](#).

Fuente de Datos	Descripción	Tipo de Dato	Herramienta / Origen de Ingesta en Azure	Tipo de Ingesta
Sensores IoT	Telemetría IoT de los campos (temperatura, humedad, pH).	Semi-estructurado (JSON)	<i>Azure IoT Hub</i> - > Azure Event Hubs	Streaming (<i>Structured Streaming</i>)
Eventos de clientes	Clics, navegación web, uso de app móvil y redes sociales.	Semi-estructurado (JSON)	<i>API App</i> -> Azure Event Hubs	Streaming (<i>Structured Streaming</i>)
Ventas online	Base de datos de la plataforma e-commerce transaccional.	Estructurado (Log de eventos CDC)	<i>Herramienta CDC (ej. Debezium)</i> -> Azure Event Hubs	Streaming (<i>Structured Streaming</i>)
Inventario local	Foto estática del stock en los almacenes (Snapshot).	Estructurado (Parquet / CSV)	<i>Exportación</i> -> Azure Data Lake Storage Gen2 (ADLS)	Batch incremental (<i>Auto Loader</i>)

Fuente de Datos	Descripción	Tipo de Dato	Herramienta / Origen de Ingesta en Azure	Tipo de Ingesta
Catálogo ERP (Productos)	Maestro de dimensiones de productos, proveedores y precios.	Estructurado (Parquet / CSV)	<i>Exportación ERP -></i> Azure Data Lake Storage Gen2 (ADLS)	Batch incremental (Auto Loader)
Meteorología	Información meteorológica de una API externa. (ej. OpenWeather).	Semi-estructurado (JSON)	<i>Script Python consumiendo la API REST -></i> Azure Data Lake Storage Gen2 (ADLS).	Batch programado (Auto Loader)

Arquitectura lakehouse



Bronze

Fuente de Datos

Estrategia Bronze (External Table)

Sensores IoT

sensor_raw

Particionada por ingest_month

Fuente de Datos	Estrategia Bronze (External Table)
Eventos de clientes	events_raw Particionada por ingest_month
Ventas online	orders_raw Particionada por ingest_month
Inventario local	inventory_raw Particionada por ingest_month
Catálogo ERP (Productos)	products_raw Sin particionar (<i>Bajo volumen</i>)
Meteorología	weather_raw Sin particionar (<i>Bajo volumen</i>)

Se mantiene el control físico y seguro de los datos crudos creando las tablas como **External Location** en **Azure Data Lake Storage Gen2 (ADLS)**. La estrategia de particionar las tablas de alto volumen por mes[1] permite configurar políticas automáticas de Azure (Storage Lifecycle Management)[2]. Esto facilita la migración transparente de datos históricos hacia almacenamientos en frío (ej. *Cold Tier* o *Archive Tier*[3]), reduciendo drásticamente los costes de infraestructura.

Silver

Tabla Origen (Bronze)	Tabla Destino (Silver - Managed Table)	Data Quality Expectations	Liquid Clustering
sensor_raw	sensor_curated	Valores anómalos (ej. temperatura) → Desvío a Cuarentena	customer_id, event_ts
events_raw	events_curated	Identificadores obligatorios nulos → Desvío a Cuarentena	customer_id, event_ts
orders_raw	orders_curated (<i>UPSERT / SCD1</i>)	Falta de valores esperados en un pedido --> tabla cuarentena	
inventory_raw	inventory_curated		
products_raw	products_curated (<i>SCD Tipo 2</i>)		

Tabla Origen (Bronze)	Tabla Destino (Silver - Managed Table)	Data Quality Expectations	Liquid Clustering
weather_raw	weather_curated		field_id, forecast_ts

A partir de la capa Silver, las tablas se convierten en **Tablas Gestionadas (Managed Tables)** cediendo el control físico a Unity Catalog. Esto permite aprovechar al máximo las optimizaciones automáticas de rendimiento y mantenimiento de Databricks[4], como la compactación de archivos y el borrado de históricos.

Gold

En la última capa de la arquitectura, se construyen vistas agregadas orientadas a dominios específicos para aportar valor directo al negocio (finanzas, logística y agricultura). Estas se implementan haciendo uso de **MATERIALIZED VIEWS** [5], garantizando latencias mínimas para las herramientas de BI como Power BI.

Parte 2: Implementación del motor de ingesta

Para desarrollar el motor de ingesta se hace uso de los Declarative Automation Bundles (DABs)[6], una herramienta de Databricks que permite definir, gestionar y desplegar de forma automatizada proyectos completos de datos. Esto facilita la adopción de las mejores prácticas de ingeniería de software, como el control de versiones, el empaquetado de código y la integración/despliegue continuos (CI/CD).

El comando `databricks bundle init` genera una plantilla base con la estructura base que va a usar el proyecto:

```
./data-engineer/moduloVII > databricks bundle init
Welcome to the default Python template for Declarative Automation Bundles!

Answer the following questions to customize your project.
You can always change your configuration in the databricks.yml file later.

Note that https://adb-7405609733291131.11.azuredatabricks.net is used for initialization.
(For information on how to change your profile, see https://docs.databricks.com/dev-tools/cli/profiles.html.)

Unique name for this project [my_project]: lakehouses_ucm
Include a job that runs a notebook: yes
Include an ETL pipeline: yes
Include a sample Python package that builds into a wheel file: yes
Use serverless compute: yes
Default catalog for any tables created by this project [masteravdc001dbr]:
Use a personal schema for each user working on this project
(this is recommended, your personal schema will be 'masteravdc001dbr.alvare05'): no, I will customize the schema configuration later in databricks.yml

🌟 Your new project has been created in the 'lakehouses_ucm' directory!

To get started, refer to the project README.md file and the documentation at https://docs.databricks.com/dev-tools/bundles/index.html.

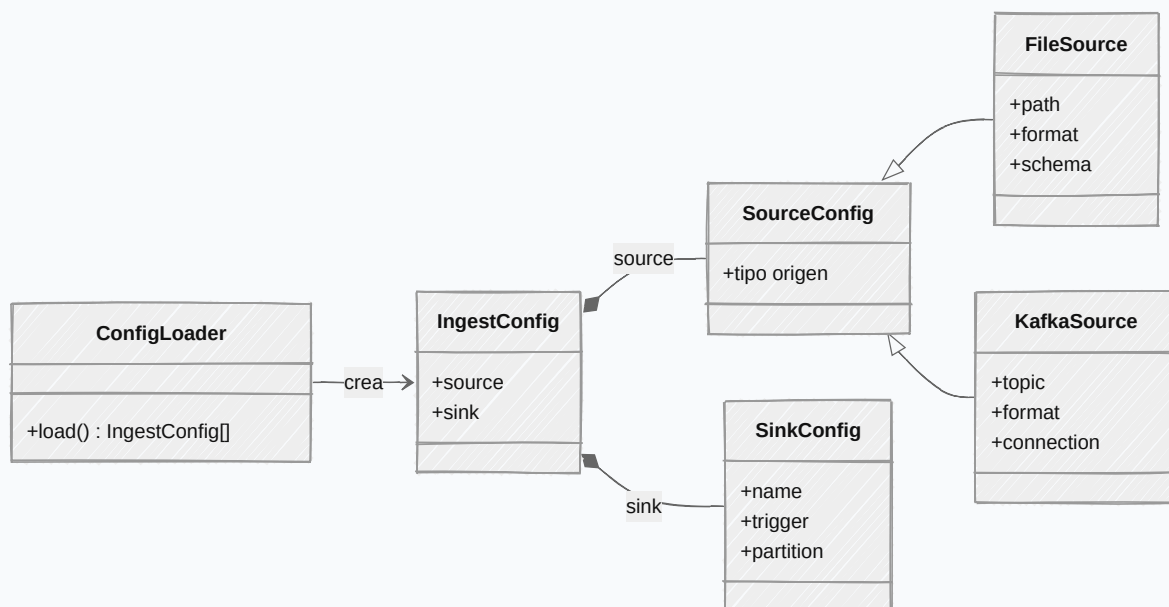
./data-engineer/moduloVII > █
```

```
my_project/
├─ databricks.yml           # Definición y configuración global del DAB
├─ pyproject.toml           # Configuración del proyecto Python y sus
                             dependencias
├─ resources/               # Definiciones de los recursos desplegados
                             por el bundle
│   └─ sample_job.job.yml   # Definición de un Databricks Job
├─ my_project_etl.pipeline.yml # Definición del pipeline ETL
├─ src/
│   └─ sample_notebook.ipynb # Notebook de ejemplo
│   └─ my_project/           # Paquete Python del proyecto
│       └─ my_project_etl/    # Código fuente del pipeline ETL
│           └─ transformations/ # Lógica de transformación del pipeline
├─ tests/                   # Tests automatizados del proyecto
└─ README.md                 # Documentación del proyecto
```

Paquete python: "motor_ingesta"

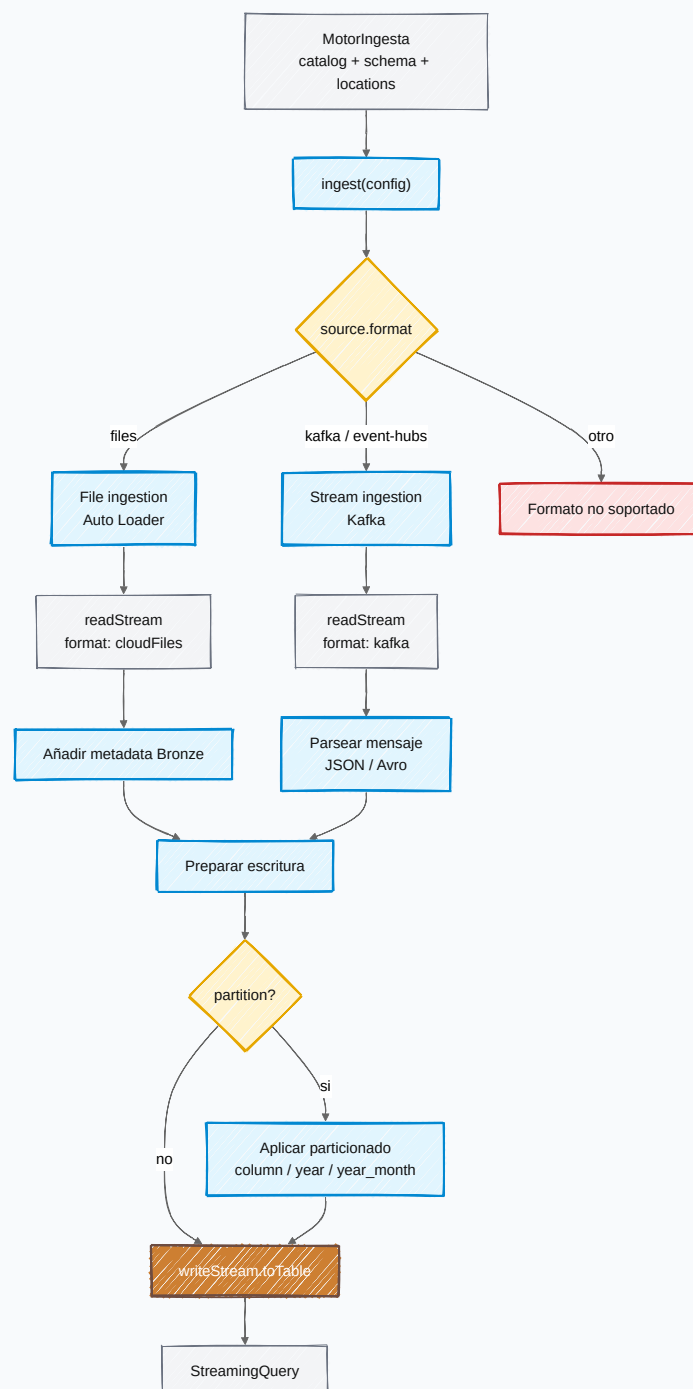
El paquete se organiza de forma modular en dos grandes bloques: un submódulo config, responsable de la definición y validación del esquema de ingesta, y la clase principal MotorIngesta, encargada de orquestar la lectura y escritura hacia la capa Bronze.

Modelo de configuración (**config/templates.py** y **config/loader.py**)



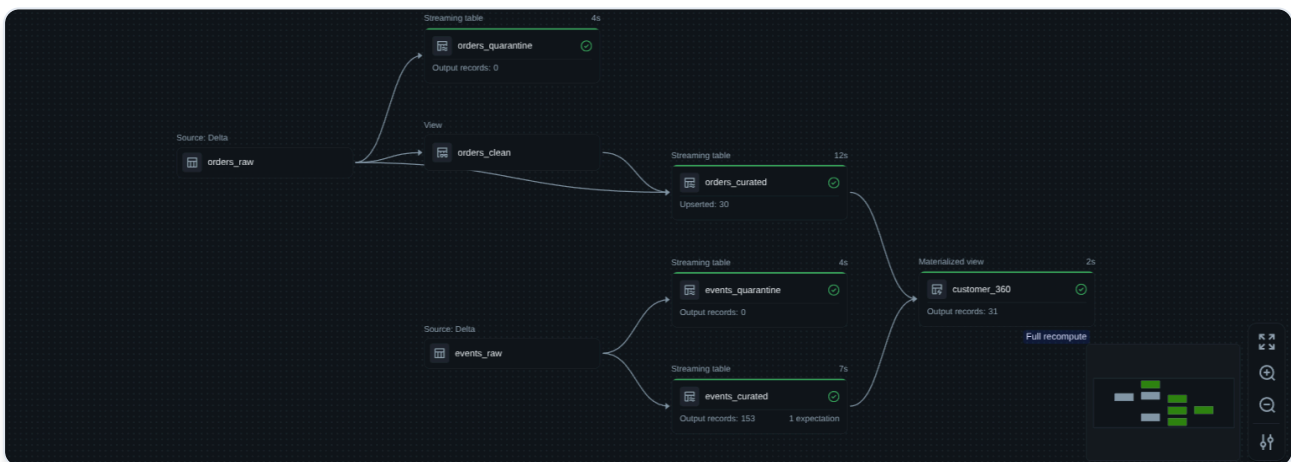
- **ConfigLoader** procesa el fichero JSON de configuración, convirtiéndolo en una lista de objetos **IngestConfig**. Este paso actúa como un contrato de datos, ya que utiliza Pydantic para validar estrictamente cada entrada (rechazando campos anómalos o combinaciones incompatibles, como el uso simultáneo de schema y schemaHints).
- Cada **IngestConfig** modela un flujo completo uniendo un origen (source: sistema de ficheros, Kafka o Event Hubs) con un destino (sink: tabla de la capa Bronze, definiendo políticas de particionado y triggers de ejecución).

Flujo de ingesta (**motor_ingesta.py** y **main.py**)



- Durante su instanciación, `MotorIngesta` configura el contexto de ejecución de Unity Catalog (USE CATALOG / USE SCHEMA) y resuelve dinámicamente las rutas de almacenamiento para esquemas y checkpoints.
- El método `ingest()` itera sobre la configuración y, según `source.format`, delega el procesamiento en métodos privados (`__file_ingestion` mediante Auto Loader/cloudFiles, o `__stream_ingestion` para Kafka/Event Hubs). En ambos casos se enriquecen los registros con metadatos de linaje (como `_ingested_at` y metadatos de origen).
- El método de escritura (`__write_stream`) aplica las lógicas de particionado y configuración de micro-batches (triggers como `availableNow` o `processingTime`) para volcar los datos en las tablas Bronze. Retorna un objeto `StreamingQuery` por fuente.
- El archivo `main.py` actúa como el punto de entrada (entry point) CLI de la aplicación[7], esperando en paralelo la resolución de todas las queries mediante un `ThreadPoolExecutor`.

Pipeline: "lakehouses_ucm_etl"



Se declara en `./resources/pipeline/lakehouses_ucm.pipeline.yml`. El pipeline implementa la evolución de los datos hacia las capas Silver y Gold.

Crea la ingesta en **Silver** y **Gold** utilizando *Spark Declarative Pipelines (SDP)*, las transformaciones se definen en: `./src/lakehouses_ucm_etl/transformations/`.

- **Calidad de datos (Capa Silver):** Se implementan data quality expectations. Los registros que violan las reglas de negocio son capturados por instrucciones como `expect_or_drop` o `expect_all_or_drop` y son desviados hacia tablas de cuarentena, asegurando la integridad de las tablas curadas mientras se retienen los datos anómalos para su auditoría.

- **Change Data Capture (CDC):** La tabla `orders_raw` captura el log de transacciones. Mediante la función `create_auto_cdc_flow` aplica un patrón Slowly Changing Dimension (SCD) Tipo 2. Esto asegura el versionado histórico de los pedidos, utilizando la columna `sequence_num` para garantizar la correcta secuenciación de los eventos.
- **Capa Semántica (Capa Gold):** La tabla `customer_360` se despliega como una Vista Materializada (Materialized View). El motor de Databricks computa y persiste físicamente las agregaciones de pedidos y eventos, y es capaz de realizar actualizaciones incrementales, que afectan únicamente a los datos subyacentes modificados en lugar de reprocesar todo el dataset.

Job: "lakehouses_ucm"



Se declara en `./resources/job/lakehouses_ucm.job.yml`.

El orquestador define un grafo de dependencias (DAG) compuesto por cuatro tareas secuenciales:

- **cargar_nuevos_datos:** Tarea inicial que ejecuta el notebook `land_new_data.ipynb` para simular o cargar nuevos ficheros en la zona de landing, usando `number_of_files` como parámetro.
- **ingesta_bronze:** Invoca el módulo Python `motor_ingesta` a través de su punto de entrada (main). Ejecuta la captura de datos hacia la capa Bronze basándose en la configuración de Unity Catalog y el JSON de ingesta.
- **etl_pipeline:** Tras la consolidación en Bronze, esta tarea lanza el pipeline DLT (`lakehouses_ucm_etl`) para procesar las reglas de negocio, calidad de datos y actualizaciones incrementales hacia las capas Silver y Gold.
- **resumen_pipeline:** Tarea final de cierre. Ejecuta un notebook de auditoría que recopila las métricas del pipeline y, en caso de éxito, dispara una notificación por correo electrónico.

Evidencias de ejecución

A continuación se incluyen vídeos que muestran el funcionamiento del paquete de ingesta y del flujo completo de procesamiento y despliegue:

Evidencia	Descripción	Vídeo
Motor de ingesta en streaming	Ejecución del motor de ingesta procesando datos en streaming.	Ver vídeo
Pipeline de ingesta	Ejecución completa del pipeline de procesamiento de datos.	Ver vídeo
Tests en Visual Studio Code	Ejecución de los tests automatizados del proyecto desde Visual Studio Code.	Ver vídeo
GitHub Actions	Ejecución del flujo de integración y despliegue definido en GitHub Actions.	Ver vídeo

Referencias y Bibliografía

- [1] Administración del ciclo de vida de los blobs (Azure Storage).
- [2] El particionado por mes se establece bajo la premisa de cumplir con las [recomendaciones de Databricks](#) de alcanzar al menos **1GB** por partición y más de **1TB** de tamaño total de tabla.
- [3] Niveles de acceso (Tiers) que ofrece Azure Blob Data.
- [4] Tablas Gestionadas vs Externas en Unity Catalog (Managed vs External).
- [5] Vistas Materializadas en Databricks: Actualizaciones incrementales para mantener eficiencia de cómputo.
- [6] Infraestructura como código: [Declarative Automation Bundles \(DAB\)](#).
- [7] Se mapea en la sección **project.scripts** del archivo `pyproject.toml`.